Embedding Gather - Video Notes

Companion to Part 2: Embedding gather. The host allocation examples describe this video's milestone. The current engine keeps shared scratch buffers in `InferenceBuffers` across prefill and decode calls; the embedding kernel's lookup and thread mapping are the same.

What this video is covering

This is the first actual model operation in the inference engine. We already did the setup work:

1. Load the Llama weights from `model.safetensors`.
2. Copy those weights into GPU memory.
3. Build CPU-side pointers that let us find a specific weight tensor quickly.
4. Tokenize a prompt into a list of token IDs.

Now we need to turn those token IDs into values the Transformer can actually do math with. That is the embedding gather.

The flow so far is:

```
input text
-> tokenizer
-> token IDs
-> copy token IDs to GPU
-> embedding gather
-> activation vectors
-> Transformer layers
```

The important distinction is that a token ID is not a meaningful number to the neural network. Token ID `9906` is not numerically similar to token ID `9907` just because the integers are close. The IDs are labels assigned by the tokenizer. We need to use each ID as an address into the model's learned embedding table.

The embedding table

Our Llama 3.2 1B model describes this tensor as:

```
(embed_tokens): Embedding(128256, 2048)
```

That gives us the shape:

```
[128256 rows, 2048 columns]
```

- There are `128256` possible token IDs in the vocabulary.
- Every token ID gets one row.
- A row has `2048` BF16 values.
- That row is called the token's embedding vector.

Think of this table as a learned dictionary. The tokenizer says, "this piece of input is token 9906." The embedding table says, "the model's learned starting representation for token 9906 is this exact 2048-number vector."

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

The values were learned in training. We do not calculate them during inference. Our job is just to find the right row quickly and move it into the activation buffer.

For a prompt with `T` tokens, the shapes are:

```
token IDs:       [T]
embedding table: [128256, 2048]
activations:     [T, 2048]
```

Example with a small fake table:

```
token IDs = [2, 0]

embedding table
row 0: [0.1, 0.2, 0.3, 0.4]
row 1: [0.5, 0.6, 0.7, 0.8]
row 2: [0.9, 1.0, 1.1, 1.2]

activations after gathering
row 0: [0.9, 1.0, 1.1, 1.2]
row 1: [0.1, 0.2, 0.3, 0.4]
```

We are not multiplying anything here. We are selecting rows. This is why it is called an embedding lookup or an embedding gather.

Where the embedding table comes from

The model loader already copied all raw tensors from the SafeTensors file into one large GPU allocation. Then it creates direct pointers to the individual tensors we need.

```
weights.embed_tokens = reinterpret_cast<__nv_bfloat16 *>(
    base + offsets.at("model.embed_tokens.weight"));
```

`base` is the start of the whole model-weight allocation on the GPU. The SafeTensors header gave us the byte offset of `model.embed_tokens.weight`. Adding the offset to the base gives us a pointer directly to the first BF16 value in the embedding table.

This matters because the table stays on the GPU after startup. We should not copy 128,256 by 2,048 embedding weights from CPU to GPU for every prompt. That would be a ridiculous amount of repeated transfer. We load weights once, then reuse them for every inference request.

Prefill setup on the host

`prefill` is the host-side function that gets a full prompt ready for model computation.

1. Allocate space for token IDs on the GPU

```
int *token_id_gpu = nullptr;

cudaMalloc(&token_id_gpu, token_ids.size() * sizeof(int));
```

`token_ids` is a CPU `std::vector<int>`. `cudaMalloc` reserves enough VRAM for the same number of `int` values and puts the starting GPU address into `token_id_gpu`.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

2. Copy token IDs from host to device

```
cudaMemcpy(
    token_id_gpu,
    token_ids.data(),
    token_ids.size() * sizeof(int),
    cudaMemcpyHostToDevice);
```

The GPU needs the IDs because the kernel will read them. `token_ids.data()` is the CPU source, `token_id_gpu` is the GPU destination, and `cudaMemcpyHostToDevice` makes the direction explicit.

The token IDs are tiny compared with the weights. Copying a short prompt's IDs is cheap. Copying the massive embedding table every time would not be.

3. Allocate output activations

```
__nv_bfloat16 *activations_gpu;

cudaMalloc(
    &activations_gpu,
    token_ids.size() * 2048 * sizeof(__nv_bfloat16));
```

For every token we need 2,048 BF16 output values. This allocation is where embedding gather will write its result.

The output is called an activation buffer because it contains the model's current working values. The embedding table contains fixed learned weights. Activations are produced from the current prompt and change for every prompt.

4. Launch the gather

```
launchEmbeddingGather(
    token_id_gpu,
    weights.embed_tokens,
    activations_gpu,
    token_count);
```

At this point every input needed by the kernel is in VRAM:

- `token_id_gpu` contains the input IDs.
- `weights.embed_tokens` points to the fixed model weights.
- `activations_gpu` is writable output storage.
- `token_count` describes the input shape.

The next operation in the current engine is RMSNorm. It reads these activation vectors and normalizes each token's 2,048 values before the first attention layer.

What a CUDA kernel is

A regular C++ function runs on the CPU. In simple terms, it runs one instruction stream at a time, although modern CPUs can still do a few things in parallel.

A CUDA kernel is a function we launch on the GPU. The GPU runs many copies of that same function at the same time, using many lightweight threads. Each thread needs a small independent piece of work.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

For embedding gather, the output contains:

```
token_count * 2048 values
```

The simplest mapping is one thread per output value.

```
thread 0    writes output value 0
thread 1    writes output value 1
thread 2    writes output value 2
...
thread N    writes output value N
```

Each thread only needs to read one token ID, read one BF16 weight, and write one BF16 activation. No thread needs another thread's result. That is an ideal first CUDA kernel because the work is embarrassingly parallel, meaning it is so independent that there is no coordination problem yet.

The embedding gather kernel

```
__global__ void embeddingGatherKernel(const __nv_bfloat16 *embed_tokens,
                                      __nv_bfloat16 *activations_gpu,
                                      const int *token_id_gpu,
                                      int token_count)
```

`__global__` says this is a CUDA kernel. It is called by host code, but it executes on the GPU.

The arguments are pointers to GPU memory, not regular CPU arrays:

- `embed_tokens` is the read-only embedding table.
- `activations_gpu` is the output activation buffer.
- `token_id_gpu` is the read-only list of input token IDs.
- `token_count` is a normal integer passed by value.

`const` on the two input pointers is useful because it tells us, and the compiler, that this kernel should not modify those inputs.

Give each thread one output index

```
const int index = blockIdx.x * blockDim.x + threadIdx.x;
```

CUDA gives every thread three built-in variables here:

- `threadIdx.x`: this thread's location inside its block.
- `blockIdx.x`: this block's location inside the grid.
- `blockDim.x`: the number of threads in one block.

The calculation flattens the block and thread positions into one unique index. For blocks with 256 threads:

```
block 0: thread 0 to 255   -> global index 0 to 255
block 1: thread 0 to 255   -> global index 256 to 511
block 2: thread 0 to 255   -> global index 512 to 767
```

This index maps directly to the flattened output buffer.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

Count the total output values

```
const int total_values = token_count * 2048;
```

The logical activation shape is `[token_count, 2048]`. GPU memory is a flat sequence of values, so the total number of values is just rows times columns.

`2048` is the hidden size for this exact model. It is currently intentionally hard-coded. Once the engine supports model configs more generally, this should come from the model configuration rather than appearing in multiple places.

Protect against extra threads

```
if (index >= total_values)
{
    return;
}
```

We launch whole blocks of threads, but `total_values` will not always divide evenly by the block size. The launcher rounds up so that there are enough threads. That means the final block can have threads with no valid output value.

This check makes those threads exit. Without it, they could read or write beyond the allocated memory, which is a bug even if it looks like it works sometimes.

Convert the flat index back into row and column

```
int token_position = index / 2048;
const int hidden_index = index % 2048;
```

The output is logically a 2D matrix but physically a flat array. Division finds the row, and modulo finds the column.

For example, if `index` is `4099`:

```
token_position = 4099 / 2048 = 2
hidden_index   = 4099 % 2048 = 3
```

So this thread owns the fourth hidden value for the third token in the prompt.

Find the vocabulary ID for this prompt position

```
const int token_id = token_id_gpu[token_position];
```

Do not mix up `token_position` and `token_id`.

- `token_position` is where the token appears in the prompt: first, second, third, etc.
- `token_id` identifies which row in the vocabulary table to use.

For example, the third prompt position might contain token ID `9906`. We use position `2` to read the input ID, then we use `9906` to select a row in the embedding table.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

Gather one learned value

```
activations_gpu[index] = embed_tokens[token_id * 2048 + hidden_index];
```

This line is the actual embedding gather.

`token_id * 2048` moves to the start of the selected row. Adding `hidden_index` selects one value inside that row. The thread writes that BF16 value into the matching location in the activation buffer.

In matrix notation, the line means:

```
activations[token_position, hidden_index]
    = embedding_table[token_id, hidden_index]
```

The flattened index is just the low-level way to write the same idea in contiguous memory.

Why the launcher exists

The kernel is the GPU work. The launcher is regular host-side code that decides how much GPU work to start and then starts it.

```
cudaError_t launchEmbeddingGather(
    const int *token_id_gpu,
    const __nv_bfloat16 *embed_tokens,
    __nv_bfloat16 *activations_gpu,
    int token_count)
```

Keeping this as a separate function is useful for a few reasons:

1. `main.cpp` can say "run embedding gather" without knowing the CUDA launch details.
2. The launch geometry lives close to the kernel it belongs to.
3. The rest of the engine gets a normal C++ function that returns a `cudaError_t`.
4. Later we can tune block size or use CUDA streams without rewriting the prefill logic.

The launcher chooses 256 threads per block:

```
const int threads_per_block = 256;
```

There is no universal magical block size. 256 is a normal starting choice that is large enough to expose lots of parallel work but still fits the hardware's scheduling model well. Performance tuning is later. Correctness and a clear work mapping come first.

Then it calculates how many blocks are needed:

```
const int blocks =
    (total_values + threads_per_block - 1) / threads_per_block;
```

This is ceiling division using integer math. It makes sure we never launch too few threads.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

For example:

```
total values = 2,050
threads/block = 256
blocks = (2050 + 256 - 1) / 256 = 9
threads launched = 9 * 256 = 2,304
```

There are 254 extra threads. That is why the bounds check in the kernel is necessary.

Finally we launch the kernel:

```
embeddingGatherKernel<<<blocks, threads_per_block>>>(
    embed_tokens,
    activations_gpu,
    token_id_gpu,
    token_count);
```

The triple angle brackets are CUDA launch syntax:

```
kernel<<<number of blocks, threads per block>>>(arguments)
```

This line does not mean the CPU waits for all GPU work to finish. CUDA kernel launches are normally asynchronous. `cudaGetLastError()` checks whether the launch itself was accepted. A later `cudaDeviceSynchronize()` is where the CPU waits and can surface an execution error from the GPU.

Why this kernel maps well to the GPU

This is mostly a memory movement operation. For each output value, we read one BF16 value and write one BF16 value. There is very little math.

For one embedding row, adjacent threads access adjacent hidden values:

```
thread 0 -> row[token ID][0]
thread 1 -> row[token ID][1]
thread 2 -> row[token ID][2]
...
```

Since the block size of 256 divides the hidden size 2048, each block stays within one 2,048-value embedding row. This is a clean contiguous access pattern. GPUs prefer nearby threads accessing nearby memory because hardware can combine those requests efficiently. This is called coalesced memory access.

The more complicated operations later will need threads to share partial results and synchronize. This gather kernel does not. Every output position is independent, so it is a good first step before RMSNorm, reductions, matrix multiplication, and attention.

What embedding gather does not do

Embedding gather gives every token its starting learned vector. It does not understand the rest of the prompt yet.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

If the same token appears twice, its raw embedding row is the same both times. Context gets added later when the transformer layers use attention to let every token combine information from earlier tokens.

The embedding table gives a starting point. The transformer turns those starting vectors into context-aware hidden states.

End state of this video

At the end, the important mental model should be:

```
Tokenizer creates IDs.
IDs are addresses, not model input values by themselves.
Embedding gather uses every ID to select one learned 2048-value row.
The gathered rows become activations on the GPU.
Those activations are the input to the first Transformer operation.
```

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.